



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №9
із дисципліни *«Технології розроблення програмного забезпечення»*
Взаємодія компонентів системи
Аудіоредактор

Виконала
студентка групи ІА–34
Кузьменко В. С.

Перевірів
викладач
Мягкий М. Ю.

Київ 2025

ЗМІСТ

Тема роботи	3
Теоретичні відомості	4
Хід роботи	5
Клієнт-серверна взаємодія	6
Архітектура аудіоредактора	7
Код системи	10
Графічний інтерфейс користувача.....	35
Висновок.....	40
Відповіді на теоретичні питання.....	41

Тема: Взаємодія компонентів системи.

Мета: Вивчити види взаємодії додатків (Client-Server, Peer-to-Peer, Service-oriented Architecture), та реалізувати в проєктованій системі одну із архітектур.

Завдання:

- Ознайомитись з короткими теоретичними відомостями.
- Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.
- Реалізувати функціонал для роботи в розподіленому оточенні відповідно до обраної теми.
- Реалізувати взаємодію розподілених частин:
 - Для клієнт-серверних варіантів: реалізація клієнтської і серверної частини додатків, а також загальної частини (middleware); зв'язок клієнтської і серверної частин за допомогою WCF, TcpClient, .NET-Remoting на розсуд виконавця.
 - Для однорангових мереж: реалізація взаємодії клієнтських додатків за допомогою WCF Peer to peer channel.
 - Для SOA додатків: реалізація сервісу, що надає послуги клієнтським застосуванням; викладання сервісу в хмару або підняття у вигляді Web Service на локальній машині; використання токенів для передачі даних про автентифікації, двостороннє шифрування.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє спроектовану архітектуру. Навести фрагменти програмного коду, які є суттєвими для відображення реалізованої архітектури.

Тема роботи:

Аудіо редактор (singleton, adapter, observer, mediator, composite, client-server)
 Аудіо редактор повинен володіти наступним функціоналом: представлення аудіо даних будь-якого формату в WAVE-формі, вибір і подальші операції копіювання

/ вставки / вирізання / деформації по сегменту аудіозапису, можливість роботи з декількома звуковими доріжками, кодування в найбільш поширених форматах (ogg, flac, mp3).

Теоретичні відомості

Клієнт-серверні застосунки поділяються на **тонкі** та **товсті клієнти** залежно від того, де виконується основна частина обчислень.

У **тонких клієнтах** більшість логіки обробки даних розташована на сервері, що зменшує вимоги до клієнтського пристрою, проте збільшує навантаження на сервер. Такий підхід зручний у випадках, коли важлива безпека, централізоване керування або обробка великої кількості користувачів і даних. У **товстих клієнтах**, навпаки, значна частина обчислень виконується на стороні користувача, що знижує навантаження на сервер, але потребує потужнішого клієнтського обладнання.

Однією з поширених моделей клієнт-серверної взаємодії є **модель “підписка/видача” (publish/subscribe)**. Вона дозволяє клієнтам автоматично отримувати оновлення від сервера без необхідності постійно надсилати запити. Такі системи зазвичай мають **три рівні**: клієнтський інтерфейс, проміжну частину (middleware), яка забезпечує взаємодію, та сервер, що обробляє бізнес-логіку, зберігає й передає дані.

У **однорангових (P2P)** системах усі учасники мають однакові права, а централізований сервер відсутній. Основними труднощами таких систем є синхронізація даних між вузлами та організація пошуку активних клієнтів. Для цього застосовуються спеціальні алгоритми, наприклад, із використанням хешування чи централізованих адрес каталогів. P2P-додатки працюють за власними протоколами та форматами обміну даними.

Сервіс-орієнтована архітектура (SOA) базується на створенні застосунків із незалежних сервісів, які взаємодіють через стандартизовані інтерфейси, такі як

SOAP або **REST**. Це забезпечує масштабованість, сумісність між різними платформами та можливість повторного використання компонентів.

Програмне забезпечення як послуга (SaaS) — це модель, у якій користувач отримує доступ до програм через Інтернет без необхідності встановлення. Вона передбачає централізоване оновлення, технічну підтримку та оплату за підпискою або за фактичним використанням. SaaS-рішення зазвичай реалізуються на основі SOA та хмарних технологій із використанням **stateless-сервісів**, які не зберігають стан між запитами.

Мікросервісна архітектура — це підхід до розробки програм, за якого система складається з окремих незалежних сервісів, кожен із яких виконує певну бізнес-функцію. Кожен мікросервіс працює у власному процесі та взаємодіє з іншими через стандартизовані протоколи, наприклад **HTTP**, **WebSockets** або **AMQP**. Така архітектура забезпечує високу масштабованість, гнучкість і спрощує оновлення окремих компонентів без впливу на всю систему.

Хід роботи

1. Ознайомитись з короткими теоретичними відомостями щодо розподілених систем та способів взаємодії між їх частинами.
2. Реалізувати частину функціоналу робочої програми у вигляді класів та їх взаємодії для досягнення необхідних функціональних можливостей.
3. Реалізувати функціонал для роботи в розподіленому оточенні відповідно до обраної теми.
4. Забезпечити взаємодію клієнтської і серверної частин програми за допомогою відповідного засобу зв'язку.
5. Підготувати звіт, що містить діаграму класів розробленої архітектури та фрагменти програмного коду, які демонструють реалізований функціонал.

Клієнт-серверна взаємодія

Ознайомившись із теоретичними засадами побудови клієнт-серверних систем, було прийнято рішення реалізувати взаємодію між клієнтом і сервером відповідно до принципів клієнт-серверної архітектури. Ця модель передбачає чітке розділення обов'язків між сторонами: сервер виконує основні обчислювальні операції, здійснює обробку даних, керує файловими потоками, контролює доступ до ресурсів і забезпечує збереження результатів, тоді як клієнт виконує роль інтерфейсу користувача, через який здійснюється ініціація запитів і отримання обробленої інформації.

Подібна архітектура дозволяє досягти гнучкості, масштабованості та централізації обробки даних, що особливо важливо для додатків, де необхідна стабільна робота з великими обсягами інформації або забезпечення узгодженості результатів для багатьох користувачів одночасно. У такій системі клієнтська частина може бути реалізована як тонкий клієнт, який виконує лише функції введення, відображення та взаємодії з користувачем, тоді як усі обчислення, перевірки та логіка бізнес-процесів виконуються на сервері.

Взаємодія між клієнтом і сервером здійснюється через мережеві протоколи обміну даними, найпоширенішим із яких є HTTP, що забезпечує стандартну передачу запитів і відповідей у форматі, зручному для взаємодії між різними платформами. Для цього використовується інтерфейс вводу-виводу (I/O), який дозволяє передавати файли, текстові або двійкові дані між сторонами системи.

Завдяки такій побудові архітектури досягається чітке розмежування відповідальності, підвищується надійність системи, а також спрощується модернізація - клієнтська частина може змінюватися незалежно від серверної. Загальну логічну структуру системи, включно з потоками даних і взаємодією між компонентами, представлено графічно на Рис. 1.

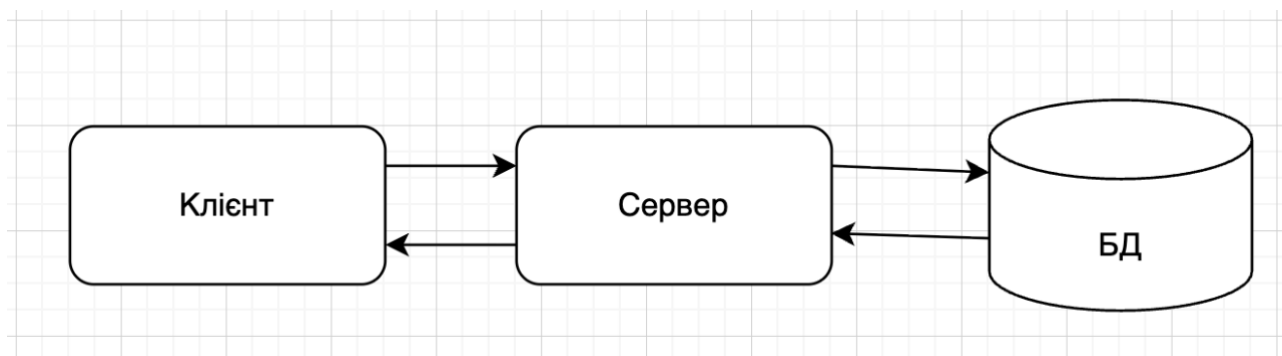


Рис. 1 – Взаємодія між компонентами

Для ініціації процесу взаємодії першочергово здійснюється запуск серверної частини застосунку. Після встановлення з'єднання клієнт підключається до сервера, який переходить у режим очікування вхідних запитів. У момент отримання запиту сервер здійснює його обробку відповідно до визначеного алгоритму та формує відповідь, яку надсилає клієнту для подальшого відображення або використання.

```

PS C:\Users\Вероніка Кузьменко\Desktop\TRP2_labs 2> & "C:\Users\Вероніка Кузьменко\Desktop\apache-maven-3.9.11\apache-maven-3.9.11\bin\mvn.cmd" -q
exec:java "-Dexec.mainClass=org.example.server.Server"
Server started on port 55555
Client connected: /127.0.0.1
  
```

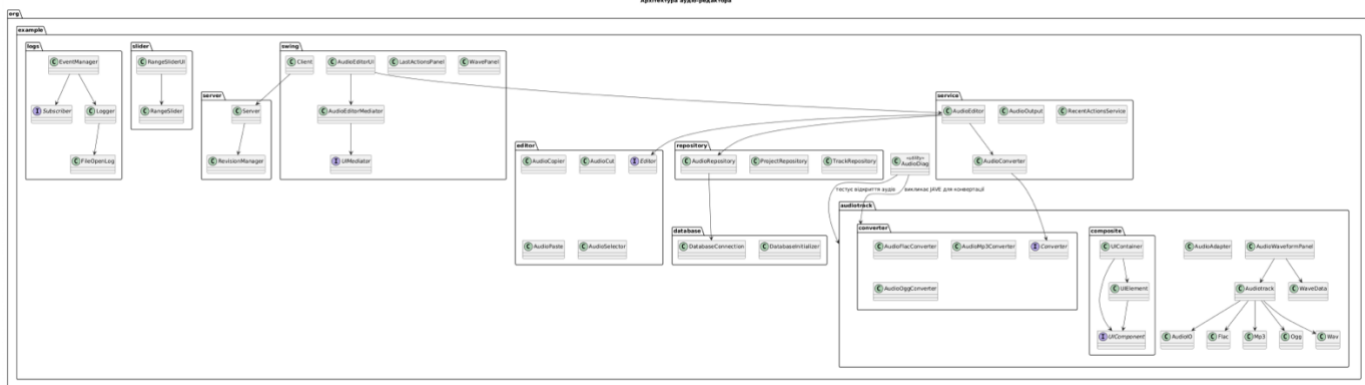
Активация Windows

Рис. 2 - Запуск серверної частини

Серверна частина застосунку ініціалізується на визначеному порту 55555, після чого переходить у режим очікування з'єднання, постійно «прослуховуючи» вхідні запити від потенційних клієнтів. Це забезпечує можливість встановлення стабільного каналу комунікації між клієнтським та серверним компонентами системи.

Архітектура аудіоредактора

Діаграма класів «Архітектура аудіоредактора» відображає структуру програмного забезпечення, розділену на пакети, кожен із яких виконує окрему функціональну роль.



У пакеті *org.example* міститься утилітний клас *AudioDiag*, який виконує тестування відкриття аудіо та взаємодію з конвертерами з інших пакетів.

Пакет `org.example.database` містить класи `DatabaseConnection` і `DatabaseInitializer`, які відповідають за підключення до бази даних та її ініціалізацію.

У пакеті *org.example.editor* зосереджено інструменти редагування аудіо. Інтерфейс *Editor* визначає базові операції редагування, а класи *AudioCut*, *AudioCopy*, *AudioPaste* і *AudioSelector* реалізують конкретні функції вирізання, копіювання, вставки та вибору частин аудіо.

Пакет *org.example.logs* реалізує систему журналювання та підписки на події. Він містить інтерфейс *Subscriber*, клас *EventManager*, що керує подіями, клас *Logger*, який записує інформацію, та *FileOpenLog*, що відповідає за логування відкриття файлів.

Пакет *org.example.repository* реалізує рівень доступу до даних, який відповідає за збереження та отримання інформації з бази даних. До нього входять класи *AudioRepository*, *ProjectRepository* та *TrackRepository*, що забезпечують роботу з даними аудіофайлів, проєктів і треків відповідно. Таке рішення ізолює логіку збереження даних від бізнес-логіки програми, що підвищує зручність у тестуванні та масштабуванні системи.

Пакет *org.example.server* містить класи *Server* і *RevisionManager*, що відповідають за роботу серверної частини програми та керування версіями проєктів.

У пакеті *org.example.service* зосереджено основну бізнес-логіку. Клас *AudioEditor* реалізує основні функції редагування і використовує класи *AudioConverter*, *AudioOutput* та *RecentActionsService*. Клас *AudioConverter* працює з інтерфейсом *Converter* для здійснення конвертації аудіо.

Пакет *org.example.slider* включає класи *RangeSlider* і *RangeSliderUI*, які реалізують графічний елемент повзунка для вибору діапазону.

Пакет *org.example.swing* реалізує графічний інтерфейс користувача. Тут визначено інтерфейс *UIMediator* і класи *AudioEditorMediator*, *AudioEditorUI*, *Client*, *LastActionsPanel* і *WavePanel*. Вони забезпечують взаємодію користувача з програмою, зв'язок між інтерфейсом і логікою через патерн «Посередник» (*Mediator*), а також підключення клієнта до сервера.

Основні зв'язки між компонентами відображають послідовність взаємодії: *AudioEditorUI* взаємодіє з *AudioEditorMediator*, який реалізує інтерфейс *UIMediator*. Клас *AudioEditorUI* використовує *AudioEditor*, що, у свою чергу,

звертається до *AudioConverter*, *Editor* та *AudioRepository*. Репозиторій взаємодіє з базою даних через *DatabaseConnection*. Класи аудіо-треків (*Audiotrack*, *AudioWaveformPanel*, *AudioIO*) зв'язані між собою та з конкретними форматами аудіо. Логування подій реалізовано через взаємодію *EventManager*, *Logger* і *FileOpenLog*. Серверна частина поєднує *Client*, *Server* і *RevisionManager*.

У підсумку, діаграма відображає багаторівневу архітектуру аудіо-редактора з чітким розподілом відповідальностей: графічний інтерфейс користувача, логіка редагування, робота з аудіо-даними, конвертація, база даних, логування та серверна взаємодія. Така структура забезпечує модульність, розширюваність і підтримуваність системи.

Код системи

Лістинг 1 - Server

```
package org.example.server;

import org.example.audiotrack.WaveData;

import javax.sound.sampled.*;
import java.io.*;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.Locale;
import java.util.Objects;

import ws.schild.jave.Encoder;
import ws.schild.jave.EncoderException;
import ws.schild.jave.MultimediaObject;
import ws.schild.jave.encode.AudioAttributes;
import ws.schild.jave.encode.EncodingAttributes;

public class Server {

    private static File sourceFile;
    private static File workingFile;
    private static File clipboardFile;

    public static void main(String[] args) {
        try (ServerSocket serverSocket = new ServerSocket(55555)) {
            System.out.println("Server started on port 55555");

            try (Socket clientSocket = serverSocket.accept();
                ObjectInputStream ois = new
ObjectInputStream(clientSocket.getInputStream());
                ObjectOutputStream oos = new
ObjectOutputStream(clientSocket.getOutputStream())) {

                System.out.println("Client connected: " +
clientSocket.getInetAddress());
```

```

while (true) {
    String cmd = (String) ois.readObject();
    if (cmd == null) continue;

    if (Objects.equals(cmd, "select")) {
        sourceFile = (File) ois.readObject();
        try {
            workingFile = prepareWorkingFile(sourceFile);

            if (workingFile == null || !workingFile.exists() ||
workingFile.length() < 1024) {
                oos.writeObject("Помилка відкриття:
FFmpeg/JavaSound створили порожній WAV. " +
                                "Спробуйте інший файл або перезapiшіть
його іншим енкодером.");
            } else {
                int[] data = new
WaveData().extractAmplitudeFromFile(workingFile);
                if (data == null || data.length == 0) {
                    oos.writeObject("Не вдалося прочитати WAV (0
семплів). " +
                                    "Можливо, нестандартні
параметри/заголовки.");
                } else {
                    RevisionManager.startSession(sourceFile,
workingFile);
                    oos.writeObject(data);
                }
            }
        } catch (Exception ex) {
            ex.printStackTrace();
            oos.writeObject("Не вдалося відкрити файл: " +
ex.getMessage());
        }
        oos.flush();

    } else if (Objects.equals(cmd, "copy")) {
        int l = (int) ois.readObject();
        int u = (int) ois.readObject();
        try {
            long[] actual = copyAudioSafe(l, u);
            int[] data = new
WaveData().extractAmplitudeFromFile(workingFile);
            RevisionManager.saveRevision("copy", actual[0],
actual[1], workingFile);
            oos.writeObject(data);
        } catch (Exception ex) {
            System.out.println("copyAudio error: " + ex);
            oos.writeObject("Помилка copy: " + ex.getMessage());
        }
        oos.flush();

    } else if (Objects.equals(cmd, "cut")) {
        int l = (int) ois.readObject();
        int u = (int) ois.readObject();
        try {
            long[] actual = cutAudioSafe(l, u);
            int[] data = new
WaveData().extractAmplitudeFromFile(workingFile);
            RevisionManager.saveRevision("cut", actual[0],
actual[1], workingFile);
            oos.writeObject(data);
        } catch (Exception ex) {
            System.out.println("cutAudio error: " + ex);

```

```

        oos.writeObject("Помилка cut: " + ex.getMessage());
    }
    oos.flush();

    } else if (Objects.equals(cmd, "paste")) {
        double x = (double) ois.readObject();
        try {
            long[] actual = pasteAudioSafe(x);
            int[] data = new
WaveData().extractAmplitudeFromFile(workingFile);
            RevisionManager.saveRevision("paste", actual[0],
actual[1], workingFile);
            oos.writeObject(data);
        } catch (Exception ex) {
            System.out.println("pasteAudio error: " + ex);
            oos.writeObject("Помилка paste: " +
ex.getMessage());
        }
        oos.flush();

    } else if (Objects.equals(cmd, "pasteAtSeconds")) {

        double sec = (double) ois.readObject();
        try {
            File ref = (workingFile != null &&
workingFile.exists()) ? workingFile : sourceFile;
            double[] stats = computeStatsDTO(ref);
            if (stats == null || stats.length < 5) {
                oos.writeObject("Статистика недоступна: спочатку
оберіть файл (Select...)");
                oos.flush();
                continue;
            }
            double durationSec = stats[4];
            if (durationSec <= 0) {
                oos.writeObject("Невідома тривалість файлу.");
                oos.flush();
                continue;
            }
            double nx = Math.max(0.0, Math.min(1.0, sec /
durationSec));

            long[] actual = pasteAudioSafe(nx);
            int[] data = new
WaveData().extractAmplitudeFromFile(workingFile);
            RevisionManager.saveRevision("paste@sec=" + sec,
actual[0], actual[1], workingFile);
            oos.writeObject(data);
        } catch (Exception ex) {
            System.out.println("pasteAtSeconds error: " + ex);
            oos.writeObject("Помилка pasteAtSeconds: " +
ex.getMessage());
        }
        oos.flush();

    } else if (Objects.equals(cmd, "deform")) {

        double startSec = (double) ois.readObject();
        double endSec    = (double) ois.readObject();
        double factor     = (double) ois.readObject();
        try {
            long[] actual = deformAudioSafe(startSec, endSec,
factor);
            int[] data = new
WaveData().extractAmplitudeFromFile(workingFile);

```

```

        RevisionManager.saveRevision("deform x" + factor,
actual[0], actual[1], workingFile);
        oos.writeObject(data);
    } catch (Exception ex) {
        ex.printStackTrace();
        oos.writeObject("Помилка deform: " +
ex.getMessage());
    }
    oos.flush();

    } else if (Objects.equals(cmd, "convertToMp3")) {
        handleConvertWithJave(oos, "mp3", "libmp3lame",
".mp3");
        RevisionManager.saveRevision("convert:mp3", -1, -1,
workingFile);

    } else if (Objects.equals(cmd, "convertToOgg")) {
        handleConvertWithJave(oos, "ogg", "libvorbis",
".ogg");
        RevisionManager.saveRevision("convert:ogg", -1, -1,
workingFile);

    } else if (Objects.equals(cmd, "convertToFlac")) {
        handleConvertWithJave(oos, "flac", "flac",
".flac");
        RevisionManager.saveRevision("convert:flac", -1, -1,
workingFile);

    } else if (Objects.equals(cmd, "getStats")) {
        try {
            double[] stats = computeStatsDTO(workingFile != null
? workingFile : sourceFile);
            if (stats == null) {
                oos.writeObject("Статистика недоступна: спочатку
оберіть файл (Select...).");
            } else {
                oos.writeObject(stats);
            }
        } catch (Exception ex) {
            ex.printStackTrace();
            oos.writeObject("Помилка отримання статистики: " +
ex.getMessage());
        }
        oos.flush();
    }
}

} catch (IOException | ClassNotFoundException e) {
    System.out.println(e.getMessage());
}
}

// Підготовка робочого WAV

private static File prepareWorkingFile(File src) throws Exception {
    if (src == null || !src.exists()) return null;

    String name = src.getName().toLowerCase(Locale.ROOT);
    if (name.endsWith(".wav")) return src;

    File out = new File(ensureTmpDir(), stripExt(src.getName()) + "
(working).wav");
    if (out.exists() && !out.delete()) {
        out = File.createTempFile(stripExt(src.getName()) + " (working) - ",
".wav", ensureTmpDir());
    }
}

```

```

    }

    try {
        transcodeToWav(src, out, true);
    } catch (Exception ignore) {}

    if (!out.exists() || out.length() < 4096) {
        try { out.delete(); } catch (Exception ignored) {}
        out = new File(ensureTmpDir(), stripExt(src.getName()) + "
(working).wav");
        transcodeToWav(src, out, false);
    }

    if (!out.exists() || out.length() < 4096) {
        try { out.delete(); } catch (Exception ignore) {}
        throw new IOException("FFmpeg створив надто малий WAV.");
    }

    System.out.println("[WORKING WAV] " + out.getAbsolutePath() + " (" +
out.length() + " bytes)");
    return out;
}

private static void transcodeToWav(File src, File out, boolean strict)
throws EncoderException {
    EncodingAttributes ea = new EncodingAttributes();
    ea.setOutputFormat("wav");

    if (strict) {
        AudioAttributes aa = new AudioAttributes();
        aa.setCodec("pcm_s16le");
        aa.setChannels(2);
        aa.setSamplingRate(44_100);
        ea.setAudioAttributes(aa);
    } else {
        ea.setAudioAttributes(null);
    }

    new Encoder().encode(new MultimediaObject(src), out, ea);
}

private static String stripExt(String n) {
    int i = n.lastIndexOf('.');
    return (i > 0) ? n.substring(0, i) : n;
}

private static File ensureTmpDir() {
    File dir = new File("target/tmp");
    if (!dir.exists()) dir.mkdirs();
    return dir;
}

// Операції редагування (повертають фактичний діапазон)

private static long[] copyAudioSafe(int startFrame, int endFrame) throws
Exception {
    if (workingFile == null) throw new IllegalStateException("Файл не
обраний.");
    try (AudioInputStream in = AudioSystem.getAudioInputStream(workingFile))
    {
        AudioFormat fmt = in.getFormat();
        int frameSize = Math.max(1, fmt.getFrameSize());
        long total = totalFrames(in, frameSize);
    }
}

```

```

        long s = clamp(startFrame, 0, total);
        long e = clamp(endFrame, 0, total);
        if (e <= s) throw new IllegalArgumentException("Порожній діапазон
copy.");

        skipFrames(in, s, frameSize);
        long framesToCopy = e - s;
        AudioInputStream segment = new AudioInputStream(in, fmt,
framesToCopy);

        clipboardFile = new File(ensureTmpDir(), "temp.wav");
        AudioSystem.write(segment, AudioFileFormat.Type.WAVE,
clipboardFile);
        return new long[]{ s, e };
    }
}

private static long[] cutAudioSafe(int startFrame, int endFrame) throws
Exception {
    if (workingFile == null) throw new IllegalStateException("Файл не
обраний.");

    try (AudioInputStream in1 =
AudioSystem.getAudioInputStream(workingFile);
        AudioInputStream in2 =
AudioSystem.getAudioInputStream(workingFile)) {

        AudioFormat fmt = in1.getFormat();
        int frameSize = Math.max(1, fmt.getFrameSize());
        long total = totalFrames(in1, frameSize);

        long s = clamp(startFrame, 0, total);
        long e = clamp(endFrame, 0, total);
        if (e <= s) return new long[]{ s, e };

        AudioInputStream first = new AudioInputStream(in1, fmt, s);
        skipFrames(in2, e, frameSize);
        long tailFrames = Math.max(0, total - e);
        AudioInputStream second = new AudioInputStream(in2, fmt,
tailFrames);

        SequenceInputStream seq = new SequenceInputStream(first, second);
        AudioInputStream out = new AudioInputStream(seq, fmt, s +
tailFrames);

        File tmp = new File(ensureTmpDir(), "cut_result.wav");
        AudioSystem.write(out, AudioFileFormat.Type.WAVE, tmp);
        copyFileUsingStream(tmp, workingFile);

        tmp.delete();

        return new long[]{ s, e };
    }
}

private static long[] pasteAudioSafe(double x) throws Exception {
    if (workingFile == null) throw new IllegalStateException("Файл не
обраний.");
    if (clipboardFile == null || !clipboardFile.exists())
        throw new IllegalStateException("Фрагмент для копіювання
відсутній.");

    try (AudioInputStream in1 =

```

```

AudioSystem.getAudioInputStream(workingFile);
    AudioInputStream in2 =
AudioSystem.getAudioInputStream(workingFile);
    AudioInputStream clip =
AudioSystem.getAudioInputStream(clipboardFile)) {

    AudioFormat fmt = in1.getFormat();
    int frameSize = Math.max(1, fmt.getFrameSize());
    long total = totalFrames(in1, frameSize);

    double nx = Math.min(1.0, Math.max(0.0, x));
    long insertAt = clamp(Math.round(total * nx), 0, total);

    AudioInputStream first = new AudioInputStream(in1, fmt, insertAt);

    skipFrames(in2, insertAt, frameSize);
    long tailFrames = Math.max(0, total - insertAt);
    AudioInputStream second = new AudioInputStream(in2, fmt,
tailFrames);

    long clipFrames = totalFrames(clip, Math.max(1,
clip.getFormat().getFrameSize()));
    AudioInputStream mid = new AudioInputStream(clip, fmt, clipFrames);

    SequenceInputStream seq = new SequenceInputStream(first, mid);
    SequenceInputStream seq2 = new SequenceInputStream(seq, second);
    AudioInputStream out = new AudioInputStream(seq2, fmt, insertAt +
clipFrames + tailFrames);

    File tmp = new File(ensureTmpDir(), "paste_result.wav");
    AudioSystem.write(out, AudioFileFormat.Type.WAVE, tmp);
    copyFileUsingStream(tmp, workingFile);

    tmp.delete();

    return new long[]{ insertAt, insertAt + clipFrames };
}
}

private static long[] deformAudioSafe(double startSec, double endSec, double
factor) throws Exception {
    if (workingFile == null) throw new IllegalStateException("Файл не
обраний.");
    if (factor <= 0.0 || Math.abs(factor - 1.0) < 1e-9)
        throw new IllegalArgumentException("Невалідний коефіцієнт.");

    try (AudioInputStream in0 =
AudioSystem.getAudioInputStream(workingFile)) {

        AudioInputStream in = ensurePcm16le(in0);
        AudioFormat fmt = in.getFormat();
        int frameSize = fmt.getFrameSize();
        float sampleRate = fmt.getSampleRate();
        long total = totalFrames(in, frameSize);

        long s = clamp(Math.round(startSec * sampleRate), 0, total);
        long e = clamp(Math.round(endSec * sampleRate), 0, total);
        if (e <= s) throw new IllegalArgumentException("Порожній діапазон
deform.");

        AudioInputStream inFirst =
AudioSystem.getAudioInputStream(workingFile);
        AudioInputStream first = new

```



```

AudioInputStream(ensurePcm16le(inFirst), fmt, s);

        try (AudioInputStream inMid0 =
AudioSystem.getAudioInputStream(workingFile)) {
            AudioInputStream inMid = ensurePcm16le(inMid0);
            skipFrames(inMid, s, frameSize);
            long midFrames = e - s;
            byte[] midBytes = readExactFramesToBytes(inMid, midFrames,
frameSize);

            long outFrames = Math.max(1, Math.round(midFrames / factor));
            byte[] midScaled = timeScalePcm16le(midBytes,
(int)fmt.getChannels(), factor, frameSize);

            try (AudioInputStream inTail0 =
AudioSystem.getAudioInputStream(workingFile)) {
                AudioInputStream inTail = ensurePcm16le(inTail0);
                skipFrames(inTail, e, frameSize);
                long tailFrames = Math.max(0, total - e);
                AudioInputStream tail = new AudioInputStream(inTail, fmt,
tailFrames);

                ByteArrayOutputStream midBAIS = new
ByteArrayInputStream(midScaled);
                AudioInputStream midAIS = new AudioInputStream(midBAIS, fmt,
outFrames);

                SequenceInputStream seq1 = new SequenceInputStream(first,
midAIS);
                SequenceInputStream seq2 = new SequenceInputStream(seq1,
tail);

                long totalOut = s + outFrames + tailFrames;
                AudioInputStream outAIS = new AudioInputStream(seq2, fmt,
totalOut);

                File tmp = new File(ensureTmpDir(), "deform_result.wav");
                AudioSystem.write(outAIS, AudioFileFormat.Type.WAVE, tmp);
                copyFileUsingStream(tmp, workingFile);

                tmp.delete();

                return new long[]{ s, e };
            }
        }
    }
}

private static AudioInputStream ensurePcm16le(AudioInputStream in) {
    AudioFormat src = in.getFormat();
    if (src.getEncoding() == AudioFormat.Encoding.PCM_SIGNED
        && !src.isBigEndian()
        && src.getSampleSizeInBits() == 16) {
        return in; // уже OK
    }
    AudioFormat target = new AudioFormat(
        AudioFormat.Encoding.PCM_SIGNED,
        src.getSampleRate() > 0 ? src.getSampleRate() : 44100f,
        16,
        src.getChannels(),
        src.getChannels() * 2,

```

```

        src.getSampleRate() > 0 ? src.getSampleRate() : 44100f,
        false
    );
    return AudioSystem.getAudioInputStream(target, in);
}

private static byte[] readExactFramesToBytes(AudioInputStream in, long
framesToRead, int frameSize) throws IOException {
    long bytesToRead = framesToRead * frameSize;
    byte[] buf = new byte[(int) bytesToRead];
    int off = 0;
    while (bytesToRead > 0) {
        int n = in.read(buf, off, (int) Math.min(64 * 1024, bytesToRead));
        if (n <= 0) break;
        off += n;
        bytesToRead -= n;
    }
    if (bytesToRead > 0) {
        byte[] trimmed = new byte[off];
        System.arraycopy(buf, 0, trimmed, 0, off);
        return trimmed;
    }
    return buf;
}

private static byte[] timeScalePcm16le(byte[] in, int channels, double
factor, int frameSize) {
    if (channels <= 0) channels = 1;
    int bytesPerSample = 2;
    int inFrames = in.length / frameSize;
    int outFrames = Math.max(1, (int) Math.round(inFrames / factor));

    byte[] out = new byte[outFrames * frameSize];

    for (int of = 0; of < outFrames; of++) {
        double srcPos = of * factor;
        int i0 = (int) Math.floor(srcPos);
        double t = srcPos - i0;
        int i1 = Math.min(inFrames - 1, i0 + 1);

        int base0 = i0 * frameSize;
        int base1 = i1 * frameSize;
        int baseOut = of * frameSize;

        for (int ch = 0; ch < channels; ch++) {
            int o0 = base0 + ch * bytesPerSample;
            int o1 = base1 + ch * bytesPerSample;

            short s0 = (short) ((in[o0+1] << 8) | (in[o0] & 0xFF));
            short s1 = (short) ((in[o1+1] << 8) | (in[o1] & 0xFF));
            int v = (int) Math.round((1.0 - t) * s0 + t * s1);
            if (v < Short.MIN_VALUE) v = Short.MIN_VALUE;
            if (v > Short.MAX_VALUE) v = Short.MAX_VALUE;

            int outOff = baseOut + ch * bytesPerSample;
            out[outOff] = (byte) (v & 0xFF);
            out[outOff + 1] = (byte) ((v >> 8) & 0xFF);
        }
    }
    return out;
}

```

```

    }

    // ----- Конвертація -----

    private static void handleConvertWithJava(ObjectOutputStream out,
                                                String container, String
audioCodec, String ext) throws IOException {

        File srcForEncode = (workingFile != null && workingFile.exists()) ?
workingFile : sourceFile;
        File baseForOutput = (sourceFile != null && sourceFile.exists()) ?
sourceFile : srcForEncode;

        if (srcForEncode == null || !srcForEncode.exists()) {
            out.writeObject("Помилка: спочатку оберіть файл (Select...).");
            out.flush();
            return;
        }

        try {
            String baseName = stripExt(baseForOutput.getName());
            File outFile = new File(baseForOutput.getParentFile(),
                baseName + " (converted to " + container + ")" + ext);

            AudioAttributes aa = new AudioAttributes();
            aa.setCodec(audioCodec);
            aa.setChannels(2);
            aa.setSamplingRate(44_100);
            if (!"flac".equals(container)) {
                aa.setBitRate(192_000);
            }

            EncodingAttributes ea = new EncodingAttributes();
            ea.setOutputFormat(container);
            ea.setAudioAttributes(aa);

            new Encoder().encode(new MultimediaObject(srcForEncode), outFile,
ea);

            System.out.println("[SERVER SAVE] -> " + outFile.getAbsolutePath());
            out.writeObject("OK: " + outFile.getAbsolutePath());
        } catch (EncoderException ex) {
            ex.printStackTrace();
            out.writeObject("Помилка конвертації: " + ex.getMessage());
        } catch (Throwable t) {
            t.printStackTrace();
            out.writeObject("Невідома помилка: " + t.getMessage());
        } finally {
            out.flush();
        }
    }

    // ----- Утиліти -----

    private static void skipFrames(AudioInputStream ais, long framesToSkip, int
frameSize) throws IOException {
        long bytesToSkip = framesToSkip * (long) frameSize;
        byte[] buf = new byte[(int) Math.min(64L * 1024, Math.max(4096,
frameSize * 1024))];
        while (bytesToSkip > 0) {
            int n = ais.read(buf, 0, (int) Math.min(buf.length, bytesToSkip));
            if (n < 0) break;
            bytesToSkip -= n;
        }
    }

```

```

    private static long totalFrames(AudioInputStream ais, int frameSize) throws
IOException {
    long fl = ais.getFrameLength();
    if (fl > 0) return fl;
    int available = ais.available();
    return (available > 0 && frameSize > 0) ? (available / frameSize) : 0;
}

    private static long clamp(long v, long lo, long hi) {
    return Math.max(lo, Math.min(hi, v));
}

    private static void copyFileUsingStream(File source, File dest) throws
IOException {
    try (InputStream is = new FileInputStream(source);
        OutputStream os = new FileOutputStream(dest)) {
        byte[] buffer = new byte[8192];
        int len;
        while ((len = is.read(buffer)) > 0) {
            os.write(buffer, 0, len);
        }
    }
}

// ----- Метадані -----

    private static double[] computeStatsDTO(File f) {
    if (f == null || !f.exists()) return null;
    try (AudioInputStream in = AudioSystem.getAudioInputStream(f)) {
        AudioFormat fmt = in.getFormat();
        int frameSize = Math.max(1, fmt.getFrameSize());
        long total = totalFrames(in, frameSize);
        double sr = fmt.getSampleRate();
        double ch = fmt.getChannels();
        double fs = frameSize;
        double dur = (sr > 0 ? total / sr : 0.0);
        return new double[]{ sr, ch, fs, total, dur };
    } catch (Exception ex) {
        ex.printStackTrace();
        return null;
    }
}
}

```

Лістинг 2 - Client

```

package org.example.swing;

import org.example.service.RecentActionsService;
import org.example.slider.RangeSlider;

import javax.swing.*;
import javax.swing.filechooser.FileNameExtensionFilter;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.io.File;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.net.Socket;

public class Client extends JFrame {

```

```

/* ----- мережеве з'єднання ----- */
private ObjectOutputStream objectOutputStream;
private ObjectInputStream objectInputStream;

/* ----- UI-компоненти ----- */
private JPanel content;
private JPanel waveContainer;
private WavePanel wavePanel;
private RangeSlider rangeSlider;

private JButton btnSelect;
private JButton btnCopy;
private JButton btnCut;
private JButton btnPaste;
private JButton btnConvertTo;

// Введення діапазону в секундах + Apply
private JSpinner startSecSpinner;
private JSpinner endSecSpinner;
private JButton applyBtn;

// Деформувати
private JSpinner factorSpinner;
private JButton deformBtn;

// Paste по секундах
private JSpinner pasteAtSecSpinner;
private JButton pasteAtBtn;

// Останні дії
private final RecentActionsService recent = new RecentActionsService();
private LastActionsPanel lastPanel;

// Поточна кількість точок у поточній хвилі (для слайдера)
private int currentDataLength = 0;

// Метадані завантаженого треку
private double sampleRate = 0.0;
private int channels = 0;
private int frameSize = 0;
private long totalFrames = 0;
private double durationSec = 0.0;

public Client() {
    super("Audio Editor (Client)");
    connectToServer();
    buildUi();
    bindActions();

    setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
    setSize(1200, 650);
    setLocationRelativeTo(null);
}

/* ----- з'єднання з сервером ----- */
private void connectToServer() {
    try {
        Socket socket = new Socket("localhost", 5555);
        objectOutputStream = new
ObjectOutputStream(socket.getOutputStream());
        objectInputStream = new ObjectInputStream(socket.getInputStream());
    } catch (IOException e) {
        JOptionPane.showMessageDialog(
            null,

```

```

        "Сервер не запущений (localhost:55555). Запустіть Server.",
        "Client",
        JOptionPane.ERROR_MESSAGE
    );
    System.exit(1);
}

/* ----- побудова UI ----- */
private void buildUi() {
    content = new JPanel(new BorderLayout());
    setContentPane(content);

    // Верхня панель з кнопками
    JPanel top = new JPanel(new FlowLayout(FlowLayout.LEFT));
    btnSelect    = new JButton("Select...");
    btnCopy      = new JButton("Copy");
    btnCut       = new JButton("Cut");
    btnPaste     = new JButton("Paste");
    btnConvertTo = new JButton("Convert to ▼");

    top.add(btnSelect);
    top.add(btnCopy);
    top.add(btnCut);
    top.add(btnPaste);
    top.add(btnConvertTo);

    // Спінери введення діапазону в секундах
    startSecSpinner = new JSpinner(new SpinnerNumberModel(0.00, 0.00, 0.00,
0.01));
    endSecSpinner   = new JSpinner(new SpinnerNumberModel(0.00, 0.00, 0.00,
0.01));
    applyBtn        = new JButton("Apply");

    startSecSpinner.setEditor(new JSpinner.NumberEditor(startSecSpinner,
"#0.00"));
    endSecSpinner.setEditor(new JSpinner.NumberEditor(endSecSpinner,
"#0.00"));

    Dimension spinSize = new Dimension(80,
startSecSpinner.getPreferredSize().height);
    startSecSpinner.setPreferredSize(spinSize);
    endSecSpinner.setPreferredSize(spinSize);

    top.add(Box.createHorizontalStrut(12));
    top.add(new JLabel("Start(s):"));
    top.add(startSecSpinner);
    top.add(new JLabel("End(s):"));
    top.add(endSecSpinner);
    top.add(applyBtn);

    // Параметри Deform
    factorSpinner = new JSpinner(new SpinnerNumberModel(1.25, 0.10, 4.00,
0.05)); // >1 швидше, <1 повільніше
    factorSpinner.setEditor(new JSpinner.NumberEditor(factorSpinner,
"#0.00"));
    factorSpinner.setPreferredSize(spinSize);
    deformBtn = new JButton("Deform");

    top.add(Box.createHorizontalStrut(12));
    top.add(new JLabel("Factor:"));
    top.add(factorSpinner);
    top.add(deformBtn);

    // Вставка по секундах

```

```

        pasteAtSecSpinner = new JSpinner(new SpinnerNumberModel(0.00, 0.00,
0.00, 0.01));
        pasteAtSecSpinner.setEditor(new JSpinner.NumberEditor(pasteAtSecSpinner,
"#0.00"));
        pasteAtSecSpinner.setPreferredSize(spinSize);
        pasteAtBtn = new JButton("Paste at (s)");

        top.add(Box.createHorizontalStrut(12));
        top.add(new JLabel("Paste at(s):"));
        top.add(pasteAtSecSpinner);
        top.add(pasteAtBtn);

        content.add(top, BorderLayout.NORTH);

        // Центральна частина: хвиля
        wavePanel = new WavePanel();
        waveContainer = new JPanel(new BorderLayout());
        waveContainer.add(new JScrollPane(wavePanel), BorderLayout.CENTER);
        content.add(waveContainer, BorderLayout.CENTER);

        // Праворуч: останні дії
        lastPanel = new LastActionsPanel();
        JPanel rightWrap = new JPanel(new BorderLayout());
        rightWrap.add(lastPanel, BorderLayout.CENTER);
        rightWrap.setPreferredSize(new Dimension(280, 10));
        content.add(rightWrap, BorderLayout.EAST);

        // Нижня частина: слайдер діапазону по точках
        rangeSlider = new RangeSlider();
        rangeSlider.setVisible(false);
        content.add(rangeSlider, BorderLayout.SOUTH);
    }

    /* ----- біндім обробники ----- */
    private void bindActions() {
        btnSelect.addActionListener(this::onSelect);
        btnCopy.addActionListener(e -> copy());
        btnCut.addActionListener(e -> cut());
        btnPaste.addActionListener(e -> paste());
        applyBtn.addActionListener(e -> onApplySelection());
        pasteAtBtn.addActionListener(e -> onPasteAtSeconds());
        deformBtn.addActionListener(e -> onDeform());

        // Меню конвертації
        JPopupMenu popup = new JPopupMenu();
        JMenuItem mp3 = new JMenuItem("mp3");
        JMenuItem ogg = new JMenuItem("ogg");
        JMenuItem flac = new JMenuItem("flac");
        mp3.addActionListener(e -> convertTo("Mp3"));
        ogg.addActionListener(e -> convertTo("Ogg"));
        flac.addActionListener(e -> convertTo("Flac"));
        popup.add(mp3); popup.add(ogg); popup.add(flac);

        btnConvertTo.addActionListener(e -> popup.show(btnConvertTo, 0,
btnConvertTo.getHeight()));
    }

    /* ----- допоміжно: масове ввімкнення/вимкнення UI ----- */
    private void setUiEnabled(boolean enabled) {
        btnSelect.setEnabled(enabled);
        btnCopy.setEnabled(enabled);
        btnCut.setEnabled(enabled);
        btnPaste.setEnabled(enabled);
        btnConvertTo.setEnabled(enabled);
        applyBtn.setEnabled(enabled);
    }

```

```

        deformBtn.setEnabled(enabled);
        pasteAtBtn.setEnabled(enabled);

        startSecSpinner.setEnabled(enabled);
        endSecSpinner.setEnabled(enabled);
        factorSpinner.setEnabled(enabled);
        pasteAtSecSpinner.setEnabled(enabled);
    }

    /* ----- логіка Apply (введення діапазону в секундах -> слайдер по
точках) ----- */
    private void onApplySelection() {
        double startS = ((Number) startSecSpinner.getValue()).doubleValue();
        double endS = ((Number) endSecSpinner.getValue()).doubleValue();

        String err = validateSeconds(startS, endS, durationSec);
        if (err != null) {
            JOptionPane.showMessageDialog(this, err, "Invalid segment",
JOptionPane.ERROR_MESSAGE);
            return;
        }

        int startIdx = secToIndex(startS);
        int endIdx = secToIndex(endS);

        rangeSlider.setMinimum(0);
        rangeSlider.setMaximum(currentDataLength);
        rangeSlider.setValue(startIdx);
        rangeSlider.setUpperValue(endIdx);
        rangeSlider.setVisible(true);

        recent.addSegment(startS, endS);
        lastPanel.refreshFrom(recent);

        JOptionPane.showMessageDialog(this, "Selection applied.", "OK",
JOptionPane.INFORMATION_MESSAGE);
    }

    /* ----- Деформація вибраного діапазону ----- */
    private void onDeform() {
        if (durationSec <= 0) {
            JOptionPane.showMessageDialog(this, "Спочатку завантажте файл.",
"Warning", JOptionPane.WARNING_MESSAGE);
            return;
        }
        double startS = ((Number) startSecSpinner.getValue()).doubleValue();
        double endS = ((Number) endSecSpinner.getValue()).doubleValue();
        String err = validateSeconds(startS, endS, durationSec);
        if (err != null) {
            JOptionPane.showMessageDialog(this, err, "Invalid segment",
JOptionPane.ERROR_MESSAGE);
            return;
        }
        double factor = ((Number) factorSpinner.getValue()).doubleValue();
        if (factor <= 0.0 || Math.abs(factor - 1.0) < 1e-9) {
            JOptionPane.showMessageDialog(this, "Factor має бути > 0 і ≠ 1.00.",
"Invalid factor", JOptionPane.ERROR_MESSAGE);
            return;
        }

        try {
            sendDeformCommand(startS, endS, factor);

            Object obj = objectInputStream.readObject();
            if (obj instanceof int[] data) {

```



```

        wavePanel.setData(data);
        currentDataLength = data.length;
        SwingUtilities.updateComponentTreeUI(content);

        rangeSlider.setMinimum(0);
        rangeSlider.setMaximum(data.length);
        rangeSlider.setVisible(true);

        requestStatsAndUpdateSecondsUI();

        recent.addSegment(startS, endS);
        lastPanel.refreshFrom(recent);

        JOptionPane.showMessageDialog(this, "Deform done (" +
String.format("%.2f", factor) + ").", "OK", JOptionPane.INFORMATION_MESSAGE);
    } else if (obj instanceof String s) {
        JOptionPane.showMessageDialog(this, s, "Deform failed",
JOptionPane.ERROR_MESSAGE);
    } else {
        JOptionPane.showMessageDialog(this, "Невідома відповідь від
сервера.", "Deform failed", JOptionPane.ERROR_MESSAGE);
    }
} catch (Exception ex) {
    ex.printStackTrace();
    JOptionPane.showMessageDialog(this, "Deform failed: " +
ex.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
}
}

/* ----- Paste по секундах (sec -> nx) ----- */
private void onPasteAtSeconds() {
    if (durationSec <= 0) {
        JOptionPane.showMessageDialog(this, "Спочатку завантажте файл.",
"Warning", JOptionPane.WARNING_MESSAGE);
        return;
    }
    double sec = ((Number) pasteAtSecSpinner.getValue()).doubleValue();
    if (Double.isNaN(sec) || sec < 0 || sec > durationSec + 1e-9) {
        JOptionPane.showMessageDialog(this, "Невірна позиція для вставки
(секунди).", "Invalid", JOptionPane.ERROR_MESSAGE);
        return;
    }
    double nx = Math.max(0.0, Math.min(1.0, sec / durationSec));
    try {
        sendPasteCommand(nx);

        Object obj = objectInputStream.readObject();
        if (obj instanceof int[] data) {
            wavePanel.setData(data);
            currentDataLength = data.length;
            SwingUtilities.updateComponentTreeUI(content);

            rangeSlider.setMinimum(0);
            rangeSlider.setMaximum(data.length);
            rangeSlider.setVisible(true);

            requestStatsAndUpdateSecondsUI();

            recent.addSegment(sec, Math.min(durationSec, sec + 0.01));
            lastPanel.refreshFrom(recent);

            JOptionPane.showMessageDialog(this, "Paste done.", "OK",
JOptionPane.INFORMATION_MESSAGE);
        } else if (obj instanceof String s) {
            JOptionPane.showMessageDialog(this, s, "Paste failed",

```

```

JOptionPane.ERROR_MESSAGE);
    } else {
        JOptionPane.showMessageDialog(this, "Невідома відповідь від
сервера.", "Paste failed", JOptionPane.ERROR_MESSAGE);
    }
} catch (Exception ex) {
    ex.printStackTrace();
    JOptionPane.showMessageDialog(this, "Paste failed: " +
ex.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
}
}

/* ----- валідація діапазону в секундах ----- */
private String validateSeconds(double start, double end, double maxSec) {
    if (currentDataLength <= 0 || maxSec <= 0) return "Файл не завантажено
або тривалість невідома.";
    if (Double.isNaN(start) || Double.isNaN(end)) return "Start/End повинні
бути числами.";
    if (start < 0) return "Start не може бути менше 0.";
    if (end < start) return "End не може бути менше Start.";
    if (end > maxSec + 1e-9) return "End не може перевищувати тривалість
доріжки.";
    return null;
}

/* ----- перетворення секунд <-> індекси точок ----- */
private int secToIndex(double sec) {
    if (durationSec <= 0 || currentDataLength <= 0) return 0;
    double norm = Math.min(1.0, Math.max(0.0, sec / durationSec));
    return (int) Math.round(norm * currentDataLength);
}
private double indexToSec(int idx) {
    if (currentDataLength <= 0) return 0.0;
    double norm = Math.min(1.0, Math.max(0.0, idx / (double)
currentDataLength));
    return norm * durationSec;
}

/* ----- вибір файлу ----- */
private void onSelect(ActionEvent e) {
    JFileChooser fc = new JFileChooser(new File("testsongs"));
    // Фільтри вибору
    fc.setAcceptAllFileFilterUsed(true);
    fc.addChoosableFileFilter(new FileNameExtensionFilter("Audio (wav, mp3,
ogg, flac)", "wav", "mp3", "ogg", "flac"));
    fc.addChoosableFileFilter(new FileNameExtensionFilter("WAV", "wav"));
    fc.addChoosableFileFilter(new FileNameExtensionFilter("MP3", "mp3"));
    fc.addChoosableFileFilter(new FileNameExtensionFilter("OGG", "ogg"));
    fc.addChoosableFileFilter(new FileNameExtensionFilter("FLAC", "flac"));

    if (fc.showOpenDialog(this) == JFileChooser.APPROVE_OPTION) {
        File selectedFile = fc.getSelectedFile();
        try {
            sendSelectCommand(selectedFile);

            Object obj = objectInputStream.readObject();
            if (obj instanceof int[] data) {

                if (data.length == 0) {
                    JOptionPane.showMessageDialog(this, "Сервер повернув
порожні дані",
                                "Open", JOptionPane.WARNING_MESSAGE);
                    return;
                }
            }
        }
    }
}

```

```

        wavePanel.setData(data);
        currentDataLength = data.length;
        SwingUtilities.updateComponentTreeUI(content);

        rangeSlider.setMinimum(0);
        rangeSlider.setMaximum(data.length);
        rangeSlider.setValue((int) (rangeSlider.getMaximum() *
0.5));

        rangeSlider.setUpperValue((int) (rangeSlider.getMaximum() *
0.75));

        rangeSlider.setVisible(true);

        requestStatsAndUpdateSecondsUI();

        // останні файли / «проекти»
        recent.addFile(selectedFile.toPath());
        recent.addProject(selectedFile.getName());
        lastPanel.refreshFrom(recent);

    } else if (obj instanceof String s) {
        JOptionPane.showMessageDialog(this, s, "Open failed",
JOptionPane.ERROR_MESSAGE);
    } else {
        JOptionPane.showMessageDialog(this, "Невідома відповідь від
сервера.",
                                "Open failed", JOptionPane.ERROR_MESSAGE);
    }
} catch (Exception ex) {
    ex.printStackTrace();
    JOptionPane.showMessageDialog(this, "Open failed: " +
ex.getMessage(),
                                "Error", JOptionPane.ERROR_MESSAGE);
}
}

/* ----- Копіювати / Вирізати / Вставити (після кліка на хвилі) -----
-- */
private void copy() {
    try {
        sendCopyCommand(rangeSlider.getValue(),
rangeSlider.getUpperValue());
        Object obj = objectInputStream.readObject();
        if (obj instanceof int[] data) {
            wavePanel.setData(data);
            currentDataLength = data.length;
            SwingUtilities.updateComponentTreeUI(content);
            rangeSlider.setMinimum(0);
            rangeSlider.setMaximum(data.length);
            rangeSlider.setVisible(true);

            requestStatsAndUpdateSecondsUI();

            recent.addSegment(indexToSec(rangeSlider.getValue()),
indexToSec(rangeSlider.getUpperValue()));
            lastPanel.refreshFrom(recent);

            JOptionPane.showMessageDialog(this, "Copy done.", "OK",
JOptionPane.INFORMATION_MESSAGE);
        } else if (obj instanceof String s) {
            JOptionPane.showMessageDialog(this, s, "Copy failed",
JOptionPane.ERROR_MESSAGE);
        }
    } catch (Exception ex) {
        ex.printStackTrace();
    }
}

```

```

        JOptionPane.showMessageDialog(this, "Copy failed: " +
ex.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
    }

    private void cut() {
        try {
            sendCutCommand(rangeSlider.getValue(), rangeSlider.getUpperValue());
            Object obj = objectInputStream.readObject();
            if (obj instanceof int[] data) {
                wavePanel.setData(data);
                currentDataLength = data.length;
                SwingUtilities.updateComponentTreeUI(content);
                rangeSlider.setMinimum(0);
                rangeSlider.setMaximum(data.length);
                rangeSlider.setValue((int) (rangeSlider.getMaximum() * 0.5));
                rangeSlider.setUpperValue((int) (rangeSlider.getMaximum() *
0.75));

                requestStatsAndUpdateSecondsUI();

                recent.addSegment(indexToSec(rangeSlider.getValue()),
indexToSec(rangeSlider.getUpperValue()));
                lastPanel.refreshFrom(recent);

                JOptionPane.showMessageDialog(this, "Cut done.", "OK",
JOptionPane.INFORMATION_MESSAGE);
            } else if (obj instanceof String s) {
                JOptionPane.showMessageDialog(this, s, "Cut failed",
JOptionPane.ERROR_MESSAGE);
            }
        } catch (Exception ex) {
            ex.printStackTrace();
            JOptionPane.showMessageDialog(this, "Cut failed: " +
ex.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
        }
    }

    private void paste() {
        try {
            Double nx = wavePanel.getLastClickNormX();
            if (nx == null) {
                JOptionPane.showMessageDialog(this,
                    "Клацніть на хвилі, щоб вибрати точку вставки.",
                    "Warning", JOptionPane.WARNING_MESSAGE);
                return;
            }

            sendPasteCommand(nx);

            Object obj = objectInputStream.readObject();
            if (obj instanceof int[] data) {
                wavePanel.setData(data);
                currentDataLength = data.length;
                SwingUtilities.updateComponentTreeUI(content);

                rangeSlider.setMinimum(0);
                rangeSlider.setMaximum(data.length);
                rangeSlider.setValue((int) (rangeSlider.getMaximum() * 0.5));
                rangeSlider.setUpperValue((int) (rangeSlider.getMaximum() *
0.75));

                requestStatsAndUpdateSecondsUI();

```

```

        double pos = nx * durationSec;
        recent.addSegment(pos, Math.min(durationSec, pos + 0.01));
        lastPanel.refreshFrom(recent);

        JOptionPane.showMessageDialog(this, "Paste done.", "OK",
JOptionPane.INFORMATION_MESSAGE);
    } else if (obj instanceof String s) {
        JOptionPane.showMessageDialog(this, s, "Paste failed",
JOptionPane.ERROR_MESSAGE);
    } else {
        JOptionPane.showMessageDialog(this, "Невідома відповідь від
сервера.",
            "Paste failed", JOptionPane.ERROR_MESSAGE);
    }

} catch (Exception ex) {
    ex.printStackTrace();
    JOptionPane.showMessageDialog(this,
        "Помилка вставки: " + ex.getMessage(),
        "Error", JOptionPane.ERROR_MESSAGE);
}
}

/* ----- НЕБЛОКУЮЧА КОНВЕРСІЯ (SwingWorker) ----- */
private void convertTo(String format) {
    JDialog progress = new JDialog(this, "Encoding...", false);
    JProgressBar bar = new JProgressBar();
    bar.setIndeterminate(true);
    progress.add(bar);
    progress.setSize(240, 70);
    progress.setLocationRelativeTo(this);

    setUiEnabled(false);
    progress.setVisible(true);

    SwingWorker<String, Void> worker = new SwingWorker<>() {
        @Override protected String doInBackground() throws Exception {
            sendConvertToCommand(format);
            Object ack = objectInputStream.readObject();
            return (ack == null) ? "unknown" : ack.toString();
        }
        @Override protected void done() {
            try {
                String msg = get();
                JOptionPane.showMessageDialog(Client.this, msg, "Convert",
JOptionPane.INFORMATION_MESSAGE);

                requestStatsAndUpdateSecondsUI();
            } catch (Exception ex) {
                ex.printStackTrace();
                JOptionPane.showMessageDialog(Client.this, "Convert failed:
" + ex.getMessage(),
                    "Error", JOptionPane.ERROR_MESSAGE);
            } finally {
                progress.dispose();
                setUiEnabled(true);
            }
        }
    };
    worker.execute();
}

/* ----- запит статистики (зі збереженням ручного введення Start/End) -
----- */
private void requestStatsAndUpdateSecondsUI() throws IOException,

```

```

ClassNotFoundException {
    // запам'ятаємо введені користувачем значення, щоб не збилися
    double prevStart = ((Number) startSecSpinner.getValue()).doubleValue();
    double prevEnd = ((Number) endSecSpinner.getValue()).doubleValue();
    double prevPaste = ((Number)
pasteAtSecSpinner.getValue()).doubleValue();

    sendGetStats();
    Object statsObj = objectInputStream.readObject();
    if (statsObj instanceof double[] arr && arr.length >= 5) {
        sampleRate = arr[0];
        channels = (int) Math.round(arr[1]);
        frameSize = (int) Math.round(arr[2]);
        totalFrames = (long) Math.round(arr[3]);
        durationSec = arr[4];

        // нові моделі спінерів під актуальну тривалість
        SpinnerNumberModel startModel = new SpinnerNumberModel(
            Math.max(0.00, Math.min(prevStart, durationSec)), 0.00,
durationSec, 0.01);
        SpinnerNumberModel endModel = new SpinnerNumberModel(
            Math.max(0.00, Math.min(prevEnd, durationSec)), 0.00,
durationSec, 0.01);
        SpinnerNumberModel pasteModel = new SpinnerNumberModel(
            Math.max(0.00, Math.min(prevPaste, durationSec)), 0.00,
durationSec, 0.01);

        startSecSpinner.setModel(startModel);
        endSecSpinner.setModel(endModel);
        pasteAtSecSpinner.setModel(pasteModel);

        // формати відображення
        startSecSpinner.setEditor(new JSpinner.NumberEditor(startSecSpinner,
"#0.00"));
        endSecSpinner.setEditor(new JSpinner.NumberEditor(endSecSpinner,
"#0.00"));
        pasteAtSecSpinner.setEditor(new
JSpinner.NumberEditor(pasteAtSecSpinner, "#0.00"));

        } else if (statsObj instanceof String s) {
            System.out.println("[getStats] " + s);
        }
    }

    /* ----- відправка команд на сервер ----- */
    private void sendCopyCommand(int l, int u) throws IOException {
        objectOutputStream.writeObject("copy");
        objectOutputStream.writeObject(l);
        objectOutputStream.writeObject(u);
        objectOutputStream.flush();
    }

    private void sendCutCommand(int l, int u) throws IOException {
        objectOutputStream.writeObject("cut");
        objectOutputStream.writeObject(l);
        objectOutputStream.writeObject(u);
        objectOutputStream.flush();
    }

    private void sendSelectCommand(File file) throws IOException {
        objectOutputStream.writeObject("select");
        objectOutputStream.writeObject(file);
        objectOutputStream.flush();
    }

    private void sendPasteCommand(double x) throws IOException {
        objectOutputStream.writeObject("paste");
        objectOutputStream.writeObject(x);
    }

```

```

        outputStream.flush();
    }
    private void sendDeformCommand(double startSec, double endSec, double
factor) throws IOException {
        outputStream.writeObject("deform");
        outputStream.writeObject(startSec);
        outputStream.writeObject(endSec);
        outputStream.writeObject(factor);
        outputStream.flush();
    }
    private void sendConvertToCommand(String format) throws IOException {
        String cmd = "convertTo" + format;
        outputStream.writeObject(cmd);
        outputStream.flush();
    }
    private void sendGetStats() throws IOException {
        outputStream.writeObject("getStats");
        outputStream.flush();
    }
}

/* ----- точка входу ----- */
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        Client ui = new Client();
        ui.setVisible(true);
    });
}
}

```

Лістинг 3 - AudioDiag

```

package org.example;

import javax.sound.sampled.*;
import javax.sound.sampled.spi.AudioFileReader;
import java.io.BufferedInputStream;
import java.io.File;
import java.io.IOException;
import java.nio.file.*;
import java.util.List;
import java.util.ServiceLoader;

import ws.schild.jave.Encoder;
import ws.schild.jave.EncoderException;
import ws.schild.jave.MultimediaObject;
import ws.schild.jave.encode.AudioAttributes;
import ws.schild.jave.encode.EncodingAttributes;
import ws.schild.jave.info.MultimediaInfo;

public class AudioDiag {

    public static void main(String[] args) {
        printFileTypes();
        printProviders();

        System.out.println("\nOpen attempts:");
        List.of(
            Paths.get("testsongs/mpthree.mp3"),
            Paths.get("testsongs/ogg.ogg"),
            Paths.get("testsongs/flac.flac")
        ).forEach(AudioDiag::tryOpenSmart);
    }
}

```

```

private static void printFileTypes() {
    System.out.println("File types visible:");
    for (AudioFileFormat.Type t : AudioSystem.getAudioFileTypes()) {
        System.out.println(" * ." + t.getExtension());
    }
}

private static void printProviders() {
    System.out.println("\nAudioFileReaders loaded:");
    for (AudioFileReader r : ServiceLoader.load(AudioFileReader.class)) {
        System.out.println(" * " + r.getClass().getName());
    }
}

private static void tryOpenSmart(Path p) {
    String label = p.toString();
    try (AudioInputStream ais = openWithJavaSound(p)) {
        AudioFormat f = ais.getFormat();
        System.out.printf("OK   %-22s  %s, %.1f kHz, %d-bit, ch=%d\n",
            label, f.getEncoding(), f.getSampleRate() / 1000.0,
            f.getSampleSizeInBits(), f.getChannels());
        return;
    } catch (Throwable ex) {
        System.out.printf("SPI FAIL %-18s  %s: %s\n", label,
            ex.getClass().getSimpleName(), ex.getMessage());
    }

    try {
        File in = p.toFile();
        MultimediaObject mo = new MultimediaObject(in);
        MultimediaInfo info = mo.getInfo();
        String codec = (info != null && info.getAudio() != null) ?
            info.getAudio().getDecoder() : "n/a";
        System.out.println("  JAVE sees codec: " + codec + " → decoding to
WAV and retrying via JavaSound...");

        Path tmp = Paths.get("target", "tmp_" + p.getFileName() + ".wav");
        decodeToWav(p, tmp);

        try (AudioInputStream ais2 = openWithJavaSound(tmp)) {
            AudioFormat f2 = ais2.getFormat();
            System.out.printf("OK   %-22s  %s, %.1f kHz, %d-bit, ch=%d (via
JAVE → %s)\n",
                label, f2.getEncoding(), f2.getSampleRate() / 1000.0,
                f2.getSampleSizeInBits(), f2.getChannels(), tmp);
        }
    } catch (Throwable ex2) {
        System.out.printf("JAVE FAIL %-16s  %s: %s\n", label,
            ex2.getClass().getSimpleName(), ex2.getMessage());
    }
}

private static AudioInputStream openWithJavaSound(Path p) throws Exception {
    return AudioSystem.getAudioInputStream(
        new BufferedInputStream(Files.newInputStream(p)));
}

private static void decodeToWav(Path in, Path out) throws EncoderException,
IOException {

```



```

Files.createDirectories(out.getParent());

AudioAttributes audio = new AudioAttributes();
audio.setCodec("pcm_s16le");
audio.setChannels(2);
audio.setSamplingRate(44100);

EncodingAttributes attrs = new EncodingAttributes();
attrs.setOutputFormat("wav");
attrs.setAudioAttributes(audio);

new Encoder().encode(new MultimediaObject(in.toFile()), out.toFile(),
attrs);
    }
}

```

Лістинг 4 - RevisionManager

```

package org.example.server;

import javax.sound.sampled.*;
import java.io.*;
import java.nio.file.Files;
import java.nio.file.StandardCopyOption;
import java.text.SimpleDateFormat;
import java.util.Date;

final class RevisionManager {

    private static File sessionDir = null;
    private static int counter = 0;

    private RevisionManager() {}

    public static void startSession(File baseFile, File workingWav) {
        try {
            String base = (baseFile != null ? stripExt(baseFile.getName()) :
"session");
            String ts = new SimpleDateFormat("yyyyMMdd-HH:mm:ss").format(new
Date());
            sessionDir = new File("target/revisions/" + base + "-" + ts);
            if (!sessionDir.exists()) sessionDir.mkdirs();
            counter = 0;

            saveRevision("select", -1, -1, workingWav);
        } catch (Exception ignore) {}
    }

    public static void saveRevision(String action, long startFrame, long
endFrame, File workingWav) {
        if (sessionDir == null) return;
        if (workingWav == null || !workingWav.exists()) return;

        counter++;
        String idx = String.format("%04d", counter);
        File snap = new File(sessionDir, "rev-" + idx + ".wav");
    }
}

```

```

    try {
        Files.copy(workingWav.toPath(), snap.toPath(),
StandardCopyOption.REPLACE_EXISTING);

        double sr = 0.0;
        long frames = 0L;
        double dur = 0.0;
        try (AudioInputStream in = AudioSystem.getAudioInputStream(snap)) {
            AudioFormat fmt = in.getFormat();
            int frameSize = Math.max(1, fmt.getFrameSize());
            frames = totalFrames(in, frameSize);
            sr = fmt.getSampleRate();
            dur = (sr > 0 ? frames / sr : 0.0);
        } catch (Exception ignored) {}

        File log = new File(sessionDir, "log.txt");
        try (PrintWriter pw = new PrintWriter(new FileWriter(log, true))) {
            String ts = new SimpleDateFormat("yyyy-MM-dd
HH:mm:ss").format(new Date());
            pw.printf(
                "%s | %s | start=%d | end=%d | frames=%d | sr=%.1f |
dur=%.3f | file=%s%n",
                ts, action, startFrame, endFrame, frames, sr, dur,
snap.getName()
            );
        }
    } catch (Exception ignore) {}
}

private static String stripExt(String n) {
    int i = n.lastIndexOf('.');
    return (i > 0) ? n.substring(0, i) : n;
}

private static long totalFrames(AudioInputStream ais, int frameSize) throws
IOException {
    long fl = ais.getFrameLength();
    if (fl > 0) return fl;
    int available = ais.available();
    return (available > 0 && frameSize > 0) ? (available / frameSize) : 0;
}
}

```

Графічний інтерфейс користувача

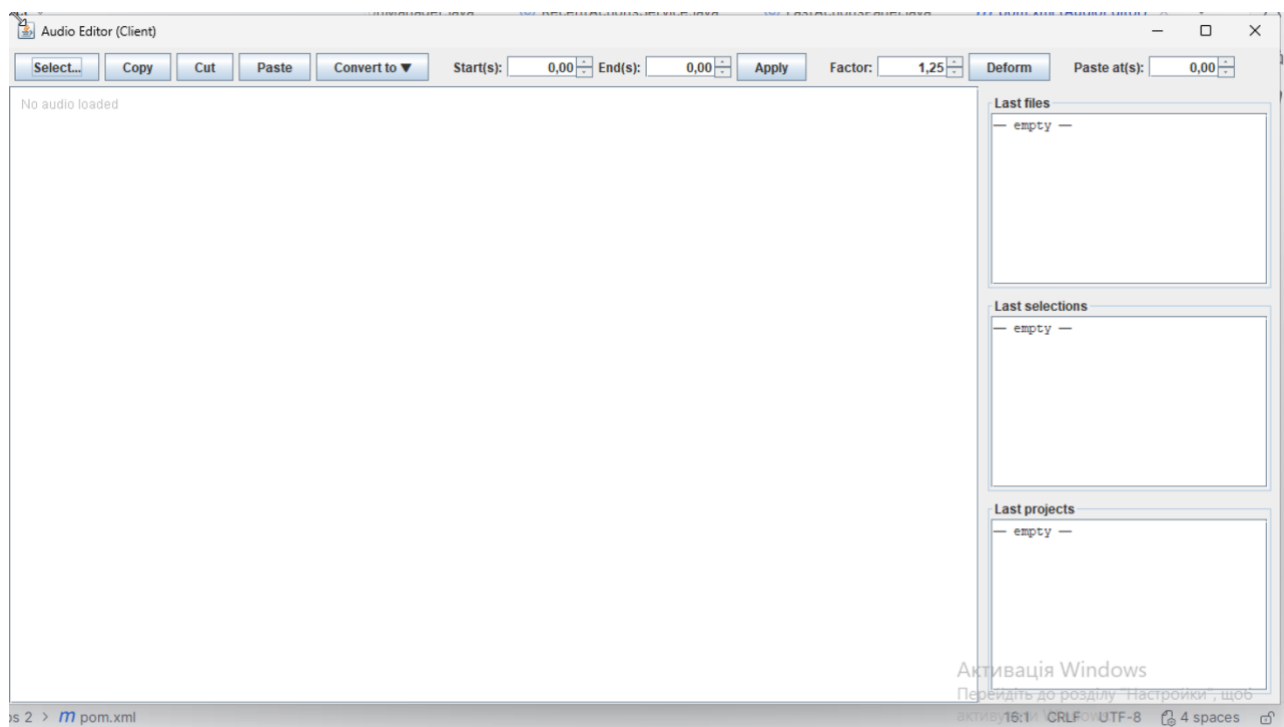


Рис. 4 – Головне меню

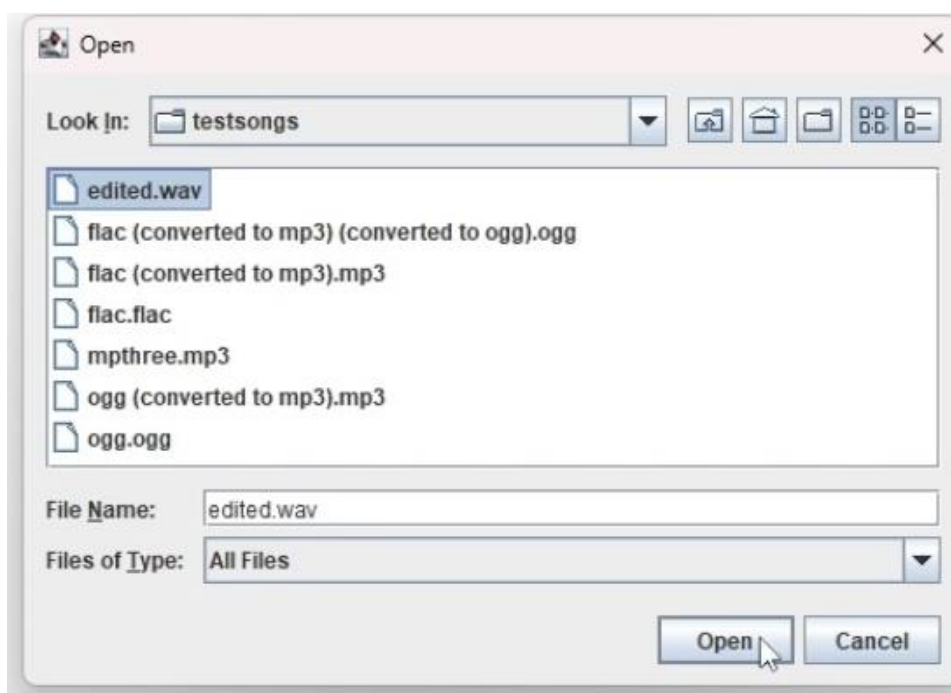


Рис. 5 – Вибір аудіо

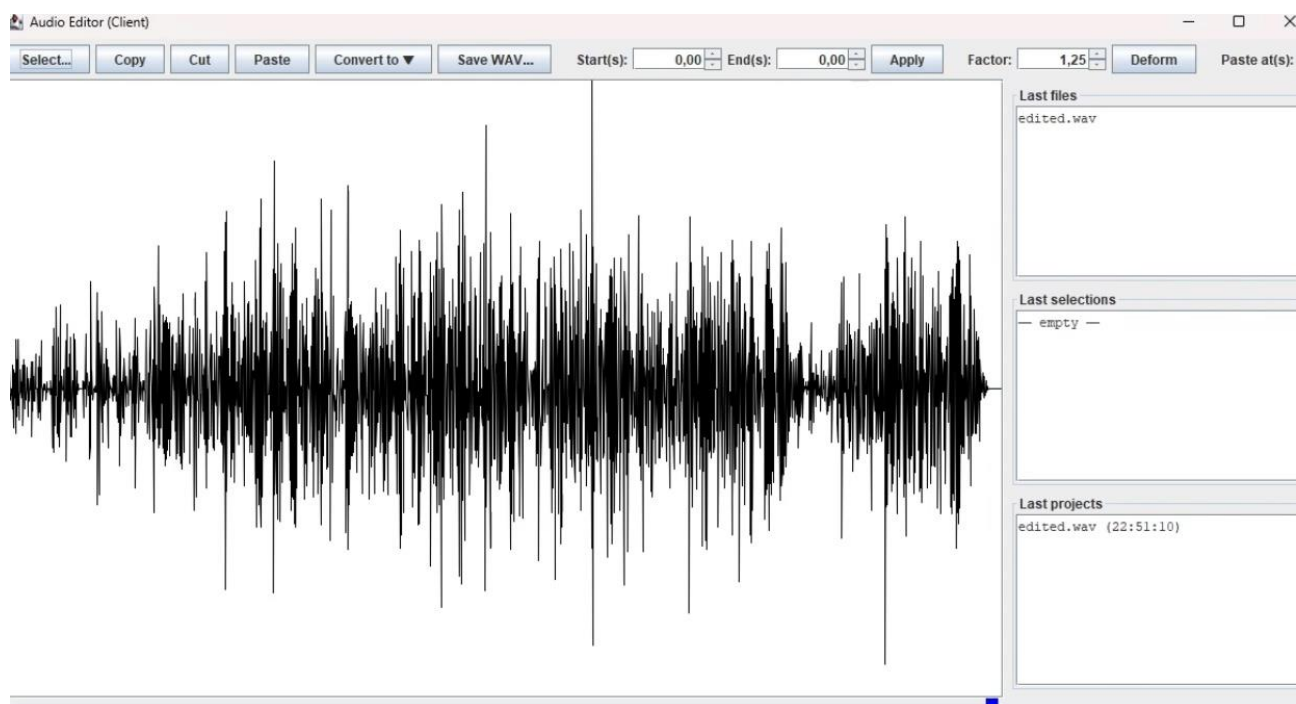


Рис. 6 – Обране аудіо для роботи

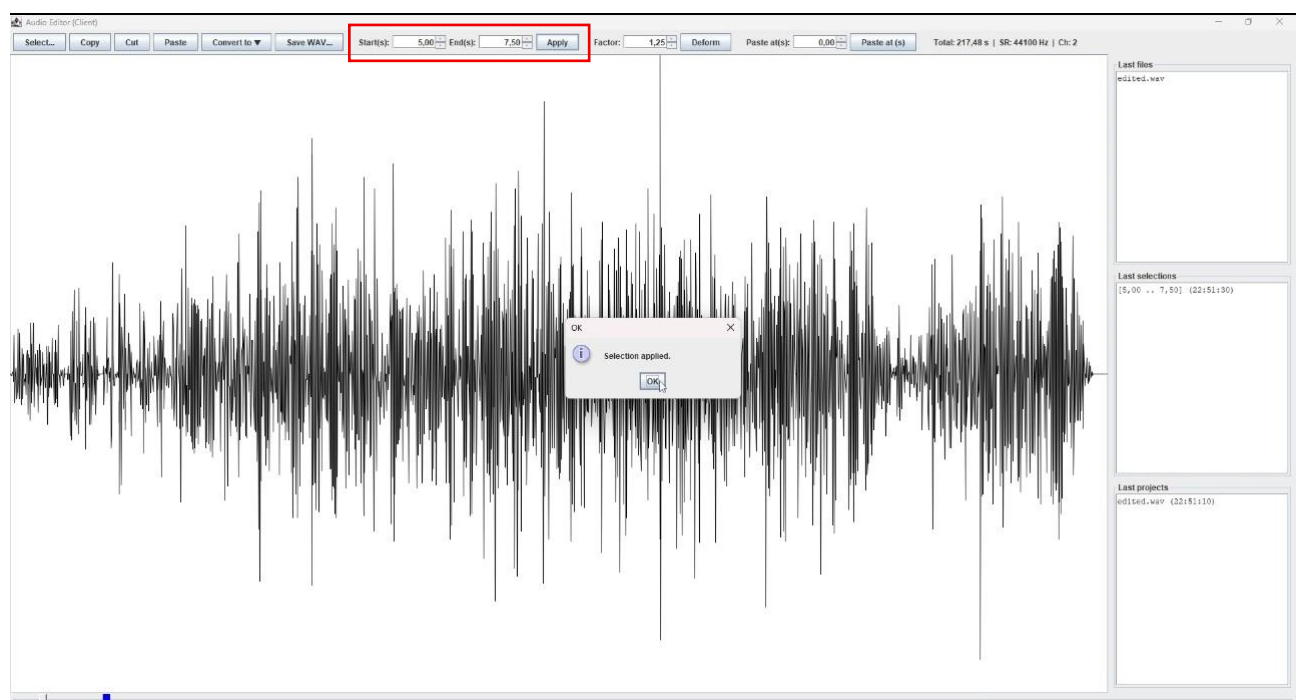


Рис. 7 – Вибір інтервалів

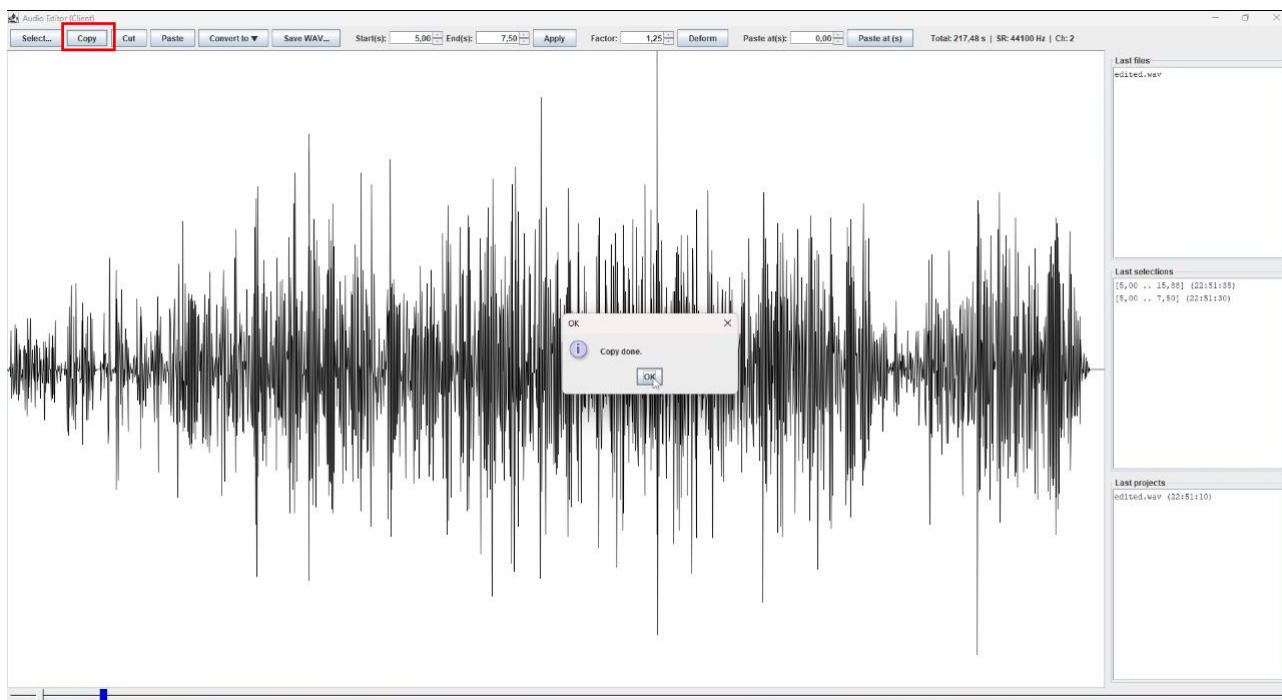


Рис. 8 – Копіювання

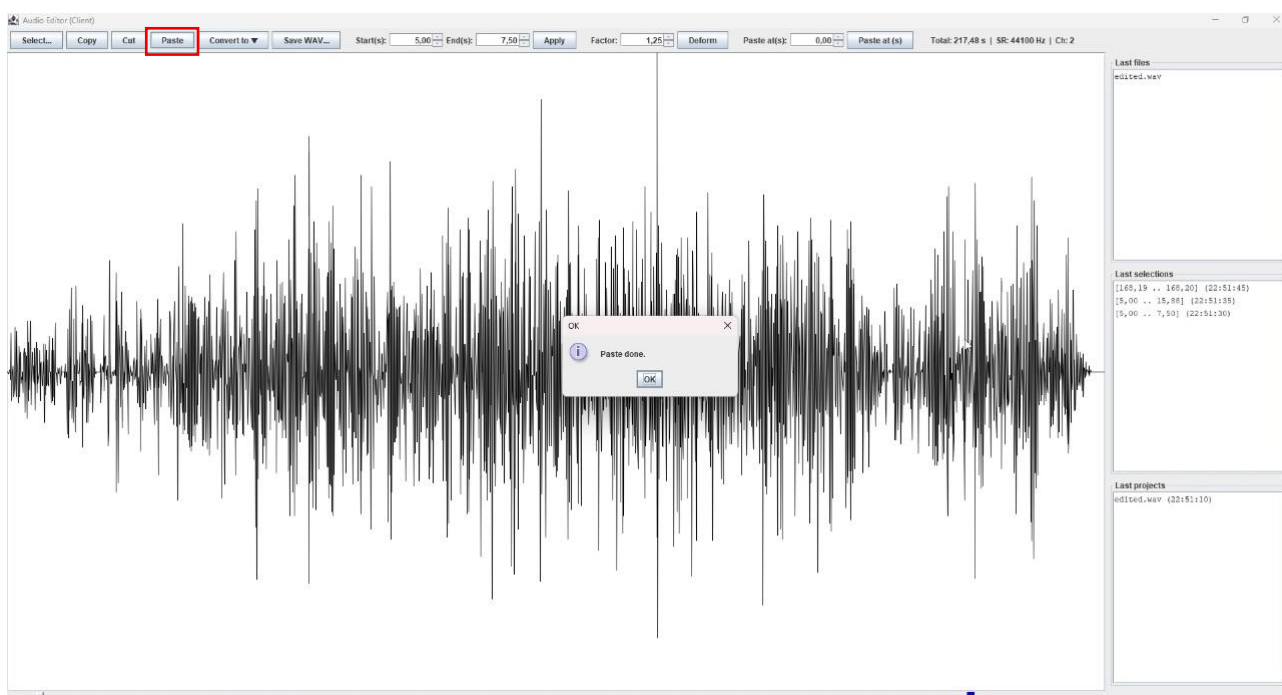


Рис. 9 – Вставка

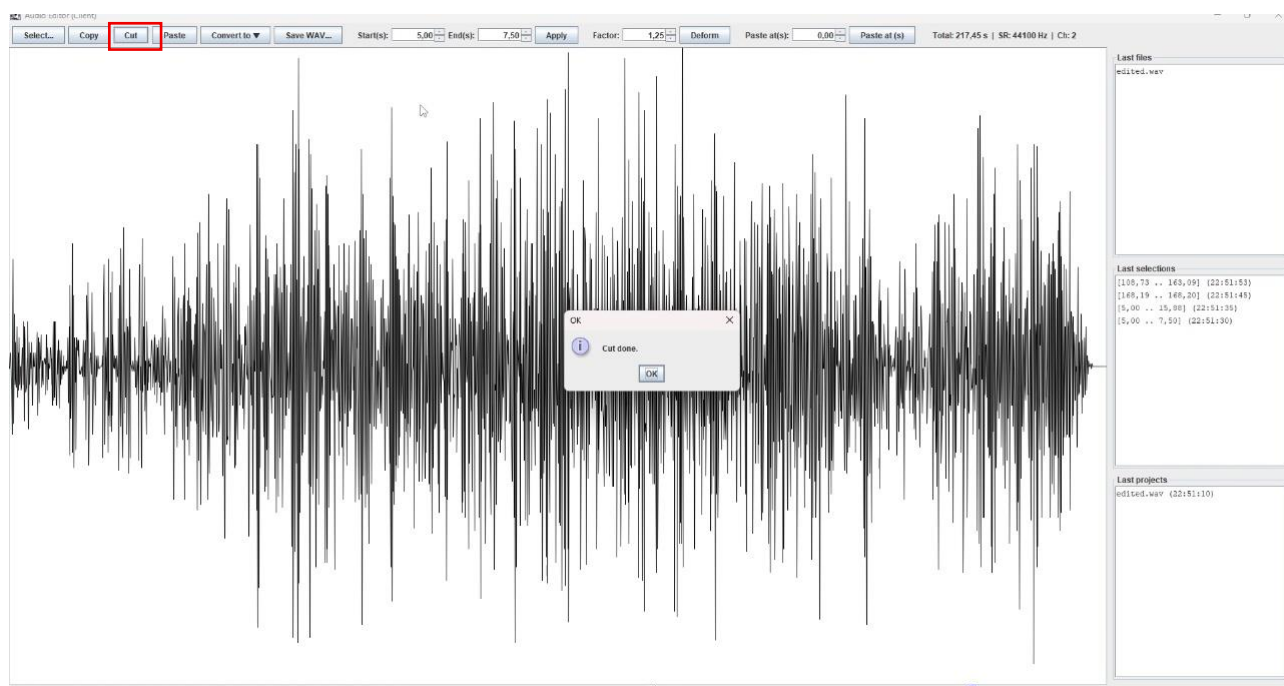


Рис. 10 – Вирізання

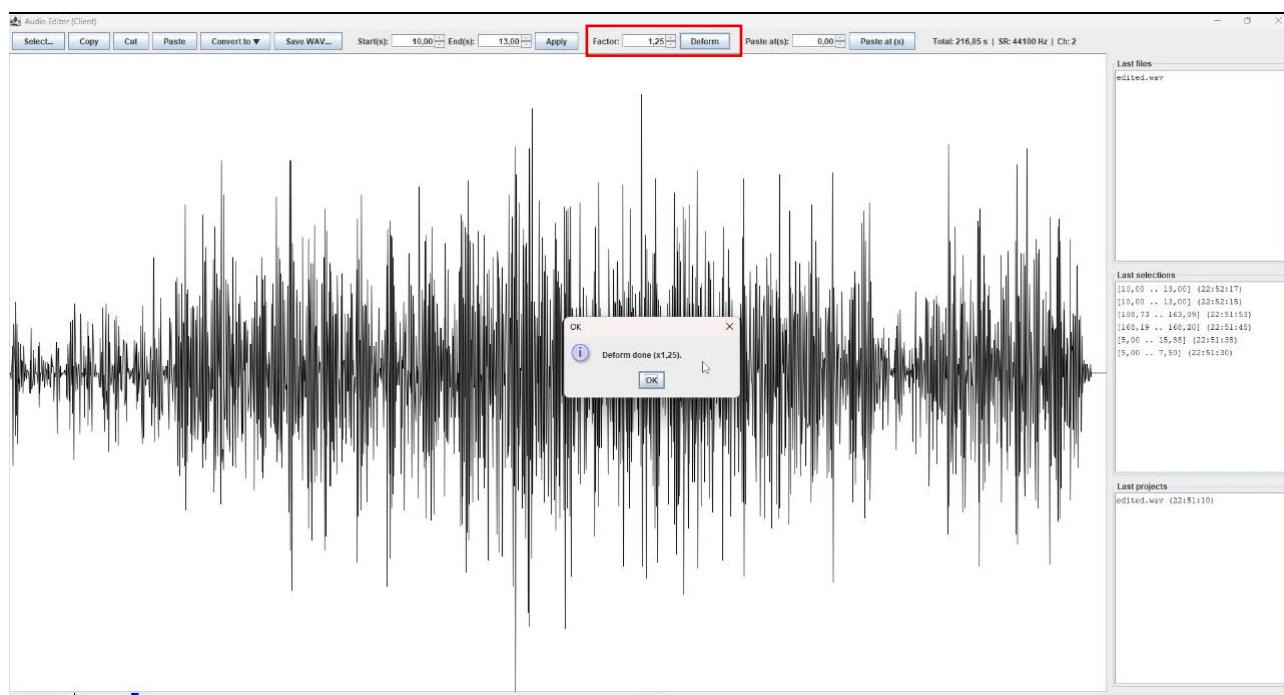


Рис. 11 – Деформація

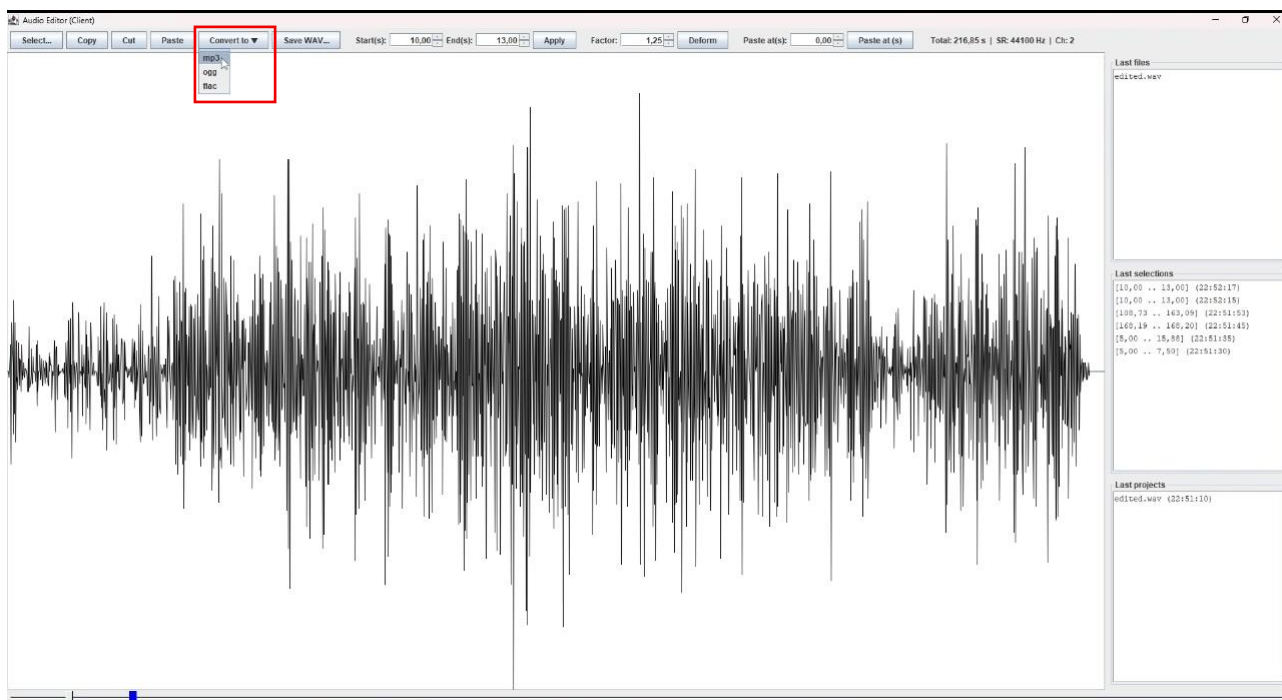


Рис. 12 – Вибір для конвертації

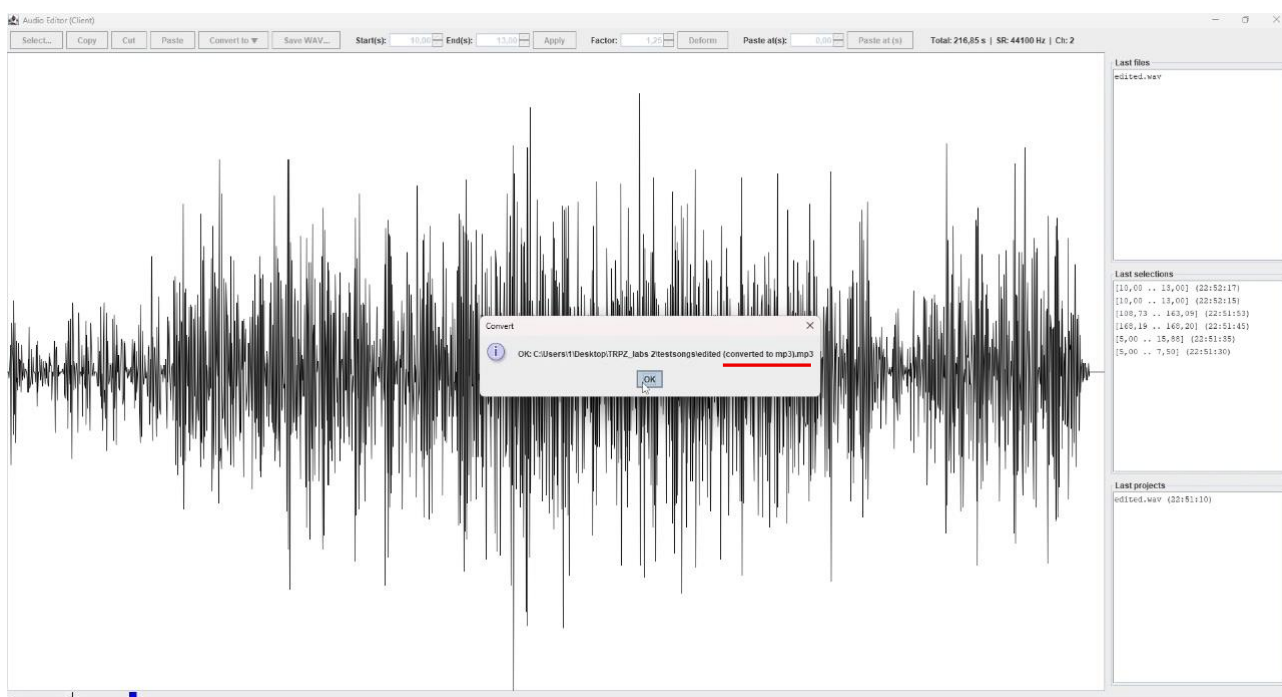


Рис. 13 – Підтвердження конвертації

Висновок

У ході виконання лабораторної роботи було вивчено ключові аспекти сучасних архітектур програмних систем, зокрема клієнт-серверної архітектури, Peer-to-Peer додатків, сервіс-орієнтованої архітектури (SOA), моделі SaaS та мікросервісного підходу. Окрему увагу було приділено організації тонких і товстих клієнтів, які визначають розподіл обчислювального навантаження між серверною та клієнтською частинами системи. Лабораторна робота дозволила закріпити теоретичні знання через практичну реалізацію клієнт-серверного додатка, що дало змогу глибше зрозуміти особливості проектування архітектури програмних систем, що забезпечують високу надійність, продуктивність і зручність використання.

Для реалізації був обраний клієнт-серверний підхід, оскільки ця архітектура забезпечує ефективну централізовану обробку даних, високий рівень продуктивності та дозволяє чітко розподілити функціональність між клієнтською та серверною частинами. Клієнт відповідає за взаємодію з користувачем і відображення даних, а сервер обробляє запити та зберігає дані, що забезпечує надійність і безпеку даних, оскільки вся інформація зберігається на сервері. Такий підхід дозволяє зменшити навантаження на клієнтські пристрої і забезпечує гнучкість у масштабуванні системи, оскільки сервер може обробляти запити від кількох клієнтів одночасно.

У процесі реалізації клієнт-серверної взаємодії було використано *TCP-сокети* через *ServerSocket* на сервері та *Socket* на клієнті, що дозволяє здійснювати обмін даними між клієнтом і сервером. Взаємодія між клієнтом і сервером здійснюється за допомогою *ObjectInputStream* та *ObjectOutputStream*, що дозволяє передавати об'єкти, зокрема аудіофайли, а також інформацію про операції, такі як копіювання, вирізання і вставка.

Крім того, для обробки аудіо використано *Java Sound API* для відкриття та аналізу аудіофайлів, а також бібліотеку *JAVE* для конвертації між різними

аудіоформатами (MP3, OGG, FLAC). Для збереження версій файлів і ведення журналу змін використано *RevisionManager*, що дозволяє зберігати копії файлів і зберігати їх історію змін.

Таким чином, виконання лабораторної роботи дозволило глибше зрозуміти принципи проектування клієнт-серверних архітектур і важливість правильної організації програмних систем. Реалізація аудіо-редактора в клієнт-серверному середовищі продемонструвала, як використання стандартних інструментів Java для створення розподілених систем дозволяє забезпечити масштабованість, продуктивність і надійність програми, що відповідає вимогам сучасних програмних рішень.

Відповіді на теоретичні питання

1. Що таке клієнт-серверна архітектура?

Клієнт-серверна архітектура - це модель взаємодії між двома основними компонентами:

- клієнтом, який надсилає запити (наприклад, користувацький застосунок або браузер);
- сервером, який отримує запити, обробляє їх і повертає результати.

Клієнт і сервер можуть працювати на різних пристроях, з'єднаних мережею. Сервер зазвичай обслуговує багато клієнтів одночасно.

2. Розкажіть про сервіс-орієнтовану архітектуру (SOA).

Сервіс-орієнтована архітектура (Service-Oriented Architecture, SOA) - це підхід до побудови програмних систем, у якому функціональність поділяється на незалежні сервіси. Кожен сервіс виконує чітко визначену бізнес-функцію (наприклад, “обробка платежу”, “реєстрація користувача”) і може використовуватись іншими частинами системи через стандартизований інтерфейс.

3. Якими принципами керується SOA?

Основні принципи SOA:

1. Повторне використання (reusability) - сервіси створюються так, щоб їх можна було використовувати у різних застосунках.
2. Слабке зв'язування (loose coupling) - сервіси мінімально залежать один від одного.
3. Контрактність (service contract) - кожен сервіс має чітко визначений інтерфейс (опис API).
4. Самодостатність (autonomy) - сервіс повинен мати власну логіку та дані.
5. Взаємодія через стандарти (interoperability) - сервіси взаємодіють через загальні протоколи (HTTP, SOAP, REST).
6. Виявлення (discoverability) - сервіси можна знайти і використати іншими компонентами.

4. Як між собою взаємодіють сервіси в SOA?

Сервіси взаємодіють через стандартизовані протоколи обміну повідомленнями, найчастіше через:

- SOAP (Simple Object Access Protocol) - формальний XML-протокол.
- REST (Representational State Transfer) — більш легкий варіант на основі HTTP.

Обмін даними зазвичай відбувається у форматах XML або JSON.

5. Як розробники взнають про існуючі сервіси і як робити до них запити?

Розробники отримують інформацію про сервіси через:

- реєстр сервісів (Service Registry) - централізоване сховище описів сервісів (наприклад, WSDL-документи для SOAP);
- API-документацію (наприклад, OpenAPI/Swagger для REST API).

Запити до сервісів робляться через мережу за допомогою стандартних методів HTTP (GET, POST, PUT, DELETE) або спеціалізованих клієнтів.

6. У чому полягають переваги та недоліки клієнт-серверної моделі?

Переваги:

- Централізоване зберігання та керування даними.
- Легше оновлювати серверну частину без змін у клієнтах.
- Підвищений рівень безпеки (через контроль доступу).
- Можливість масштабування сервера під більшу кількість клієнтів.

Недоліки:

- Залежність від сервера - якщо він виходить з ладу, система недоступна.
- Нерівномірне навантаження (сервер перевантажується, клієнти - просто очікують).
- Необхідність постійного з'єднання з мережею.

7. У чому полягають переваги та недоліки однорангової (peer-to-peer, P2P) моделі?

Переваги:

- Відсутність центрального сервера - менше ризику відмови.
- Кожен вузол може бути і клієнтом, і сервером.
- Добре підходить для розподілу великих обсягів даних (торенти, блокчейн).

Недоліки:

- Складніше забезпечити безпеку і контроль доступу.
- Важко гарантувати стабільну роботу та узгодженість даних.
- Продуктивність залежить від ресурсів і доступності кожного вузла.

8. Що таке мікросервісна архітектура?

Мікросервісна архітектура - це розвиток ідеї SOA, де система складається з малих, незалежних сервісів, кожен з яких відповідає за одну чітку функцію. Кожен мікросервіс має:

- власну базу даних,
- власний інтерфейс (зазвичай REST API),
- можливість незалежного розгортання та оновлення.

9. Які протоколи використовуються для обміну даними в мікросервісній архітектурі?

Основні протоколи:

- HTTP/HTTPS (REST API - найпоширеніший варіант);
- gRPC (ефективна двостороння комунікація через Protocol Buffers);
- AMQP / Kafka / RabbitMQ - для асинхронного обміну повідомленнями між сервісами;
- WebSocket - для постійного двостороннього з'єднання (наприклад, реального часу).

10. Чи можна назвати підхід сервіс-орієнтованою архітектурою, коли ми в проєкті між шаром веб-контролерів та шаром доступу до даних реалізуємо шар бізнес-логіки у вигляді сервісів?

Ні, це не повна SOA, а лише шарова архітектура (layered architecture) з виділенням шару сервісів. У SOA сервіси - це незалежні, розподілені компоненти, які можуть бути викликані через мережу різними застосунками. У випадку “шару сервісів” - це лише внутрішня логічна організація коду в межах одного застосунку, а не самостійні сервіси.